

A Monte Carlo Tree Search-Based Algorithm for Workflow Scheduling

Bocong Zhao¹, Mingyuan Cai², Mengbing Zhou¹, Yang Wang^{1,2,3}

¹ Shenzhen Institutes of Advanced Technology, Chinese Academy of Sciences, Shenzhen, China

² The SIAT-Suntang Big Data&AI Joint Innovation Laboratory, Shenzhen, China

³ Shenzhen University of Advanced Technology, Shenzhen, China

* Corresponding: yang.wang1@siat.ac.cn

Abstract—We present an efficient algorithm for workflow scheduling, where the resource-request graph can be modeled as a directed acyclic graph (DAG) to enable parallel execution. Our algorithm employs Monte Carlo Tree Search (MCTS) to enhance resource utilization by fully leveraging the probability distribution of task execution times, rather than relying solely on a single execution time estimate. Building on this foundation, we introduce several targeted optimizations to traditional Monte Carlo Simulation (MCS) to overcome the limitations of static scheduling strategies when addressing the uncertainty of execution makespans. We developed a prototype system, named scDAG, which is a functionally extended and enhanced version of the CloudSim framework. Through implementation and experiments on scDAG, our algorithm demonstrates a superior approach to workflow scheduling. In addition, we analyze and compare the experimental data, revealing significant improvements. Our work ultimately achieves better performance by minimizing workflow scheduling time.

Index Terms—workflow, task scheduling, directed acyclic graph, Monte Carlo Tree Search.

1. Introduction

Nowadays, traditional data processing methods are increasingly unable to satisfy the computing requirements of large-scale complex environments. To solve the performance bottleneck problem of a single computing device, large-scale distributed computing platforms such as cluster computing, grid computing [1], cloud computing, and edge computing have emerged. Regardless of the distributed computing architecture, a well-performing task scheduling algorithm is significant, as it directly influences the quality of resource utilization on the platform. As shown in Figure 1, a typical AI workflow consists of modules (tasks) such as data input, data transformation, parameter tuning, model training, model validation, and model deployment, where the dependencies between modules are realized through data flow.

Task scheduling can be divided into three types: divisible tasks [2], indivisible tasks, and workflow tasks. Divisible tasks are those that can be decomposed into any size of subtasks, which are independent of each other. In

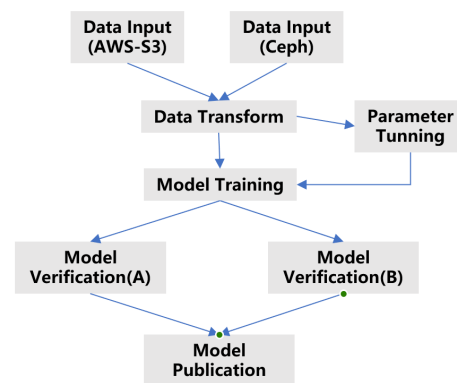


Figure 1: A typical AI model workflow.

contrast, indivisible tasks cannot be divided into subtasks. A workflow is composed of size-fixed subtasks, some of which are order-dependent, typically expressed by a Directed Acyclic Graph (DAG). It has been proven that workflow task scheduling in heterogeneous distributed systems is an NP-hard problem [3] due to the dependent relationships among subtasks and the varying performances of processors and communication links in heterogeneous environments. Workflow scheduling, as a type of task scheduling, can be divided into two steps. The first step involves resource selection and supply. In the second step, a plan is generated to map the most appropriate resources for all subtasks [4].

The Monte Carlo method [5], a statistical simulation technique, estimates statistical values of target parameters through random sampling. MCTS [6] applies this method by constructing a search tree and sampling within the decision space, as illustrated in Figure 2. The process involves four main steps: selection (traversing the tree to a leaf node), expansion (adding a child node), simulation (playing out the game from the new node), and backpropagation (updating node values based on simulation results). This iterative process refines the search tree to identify the best move or solution. While MCTS is widely used in AI, particularly in games like Go, this paper introduces a novel MCTS-based algorithm for workflow scheduling modeled as a DAG, demonstrating its feasibility and efficiency in this context.

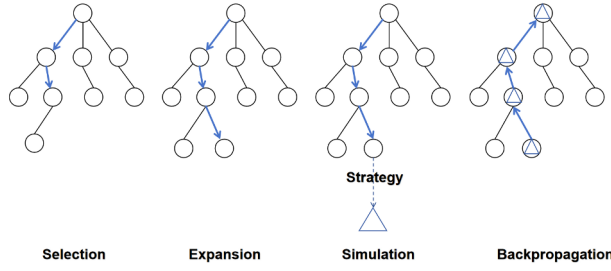


Figure 2: MCTS is a heuristic search algorithm used for decision-making in domains like games, planning, and optimization. It iteratively builds a search tree by performing the four main steps.

Traditional static scheduling has significant uncertainties, and depending on the scheduling algorithm used, it may perform poorly. Heuristic algorithms similar to MCTS are good at handling uncertainty and randomness by simulating and evaluating different scheduling strategies multiple times, allowing them to find better solutions, especially for tasks with uncertain execution times. Due to efficiency considerations, traditional static scheduling often relies on local optimal solutions. In contrast, MCTS effectively explores the global search space through random sampling and incremental tree expansion, enabling it to discover potential high-quality scheduling solutions. Moreover, MCTS can make decisions in a relatively short time, making it suitable for scheduling large-scale workflows, particularly in complex dependency and resource allocation scenarios. We treat task execution makespans as fixed values and set exact relationships between task execution durations and resources at the beginning of the scheduling process. This approach transforms the dynamically complex online problem into an approximate offline version, leveraging the probabilistic distribution characteristics of task execution times to achieve more efficient scheduling strategies and overall performance optimization. Finally, we compare our algorithm with random algorithm, HEFT and CPOP because these classical algorithms are representative and reliable.

In summary, our **contributions** are as follows:

- 1) We introduce several DAG models and make specific definitions of both variables and relationships among them. In addition, we establish an optimization problem for task scheduling with the objective of minimizing the total makespan time of the workflow.
- 2) We propose a MSA algorithms based on MCTS for workflows. MSA leverages the probability distribution characteristics of task execution times, aiming to overcome the limitations of static scheduling strategies in handling the uncertainty of task execution times.
- 3) We develop sCDAG which is based on CloudSim with extended functions and modules, to evaluate

our algorithm and compare with randomized algorithms, HEFT and CPOP [7].

In the reminders of this paper, we overview related work in Section 2, then introduce the DAG and workflow model with constraints and analyze the time complexity in Section 3. After modeling, we propose our workflow scheduling algorithm in Section 4. In Section 5, we demonstrate performance of our algorithm compared with several algorithms. We finally conclude the paper in Section 6.

2. Related Work

Workflow scheduling is a critical task in various domains, including satellite task planning, traffic flow control, toolpath planning, program tuning, job shop scheduling, microgrid scheduling, maintenance policy improvement, and mobile edge-cloud computing systems. It is a well-studied problem due to its significance in various applications [3], [8], [9]. Different types of computing platforms have led to the development of various scheduling algorithms. In the context of heterogeneous cloud computing platforms, Wang et al. [3] creatively constructed a multi-queue request scheduling framework that transformed the complex scheduling problem into a series of queue models. They predicted the overall workflow task completion time by accurately calculating the ideal response time of each queue. Similarly, Liu et al. [8] explored an online greedy scheduling algorithm that generates scheduling schemes conforming to requirements and effectively addresses challenges in practical operations.

Yang et al. [10] focused on a heterogeneous fog computing network platform, where they constructed a detailed model analyzing discharge circuits, computing resources, and task energy consumption. Based on this analysis, they proposed a method to significantly improve the energy efficiency of the task scheduling model. Li et al. [9] introduced a novel cluster scheduler aimed at minimizing task completion time, designing a corresponding heuristic method to solve the complex resource allocation problem. Nabi et al. [11] aimed to maximize processor resource utilization and presented an innovative dynamic task scheduling strategy that uses resource status information to effectively enhance system resource utilization. Moreover, Calzarossa et al. [12] focused on minimizing both task completion time and resource costs by designing a multi-objective optimization scheduling model with resource and time constraints.

Scheduling methods can be categorized into static and dynamic scheduling. Static scheduling involves creating a complete plan and resource mapping in advance based on available information and task characteristics. For instance, Huff et al. [13] proposed a resource-efficient static scheduling strategy, while Topcuoglu et al. developed the Heterogeneous Earliest-Finish-Time (HEFT) algorithm [7]. In contrast, dynamic scheduling emphasizes real-time responsiveness and adaptability, optimizing resource allocation based on current conditions. Zhou et al. [14] proposed a dynamic scheduling decision model based on ensemble forest theory,

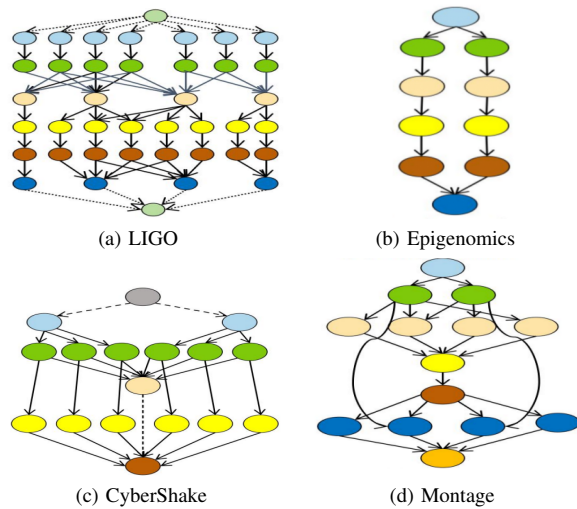


Figure 3: Four types of scientific workflows.

which can automatically select the optimal strategy among four typical dynamic scheduling strategies according to different operating conditions. Zhang et al. [15] proposed a hyper-heuristic method for multi-task scheduling inspired by genetic programming. List scheduling algorithms prioritize tasks to create an ordered schedule. For example, Arabnejad et al. [16] proposed the Predicted Earliest Finished Time (PEFT) algorithm, which uses an optimistic cost table (OCT) to calculate the minimum computing and communication costs across processors. The HEFT and CPOP algorithms use uplink and downlink ranks to determine task priorities.

Several studies have explored the application of MCTS in optimizing workflow scheduling algorithms. Eddy et al. [17] introduced a semi-Markov decision process formulation for satellite task scheduling and presented solution approaches based on forward search and MCTS. Cheng et al. [18] focused on improving the solution quality and runtime efficiency of an MCTS-based traffic flow control algorithm for arterial intersections. Yoo et al. [19] developed an algorithm for FDM toolpath planning using MCTS, demonstrating its effectiveness in navigating large search spaces. Similarly, Haj-Ali et al. [20] applied MCTS to tune programs for high-performance tasks, outperforming traditional algorithms. Li et al. [21] proposed an effective MCTS-based algorithm for minimizing makespan in dynamic flexible job shop scheduling problems. Shuai et al. [22] investigated online scheduling of a residential microgrid using MCTS and reinforcement learning without requiring a forecast model. Hoffman et al. [23] addressed the online improvement of condition-based maintenance policies using MCTS in a two-stage approach. Furthermore, Oren et al. [24] combined reinforcement learning and MCTS to enhance performance in combinatorial optimization problems. Tuli et al. [25] introduced MCDS, an AI-based scheduling approach using MCTS and deep

surrogate models for workflow applications in mobile edge-cloud computing systems. Kung et al. [26] presented an improved MCTS approach for workflow scheduling, highlighting the potential of MCTS in this domain. Overall, the literature demonstrates the versatility and effectiveness of MCTS in optimizing workflow scheduling algorithms across various applications, showcasing its potential for enhancing efficiency and performance in complex scheduling tasks.

3. Modelling

We regard the process of workflow scheduling as finding a sequence of a series of interrelated tasks, which can be described as a DAG. Figure 4 shows an typical instance of a DAG workflow, basically consisting of nodes and directed edges. We will illustrate our model from DAG model to workflow model gradually.

3.1. Common Workflow Models

In the study of DAG workflow scheduling strategies, the selection of appropriate workflow models is crucial to enhance the application value of research findings and understanding the complexity of scheduling problems. This paper investigates four DAG workflow instances from different scientific domains: the LIGO workflow, the Epigenomics workflow, the CyberShake workflow, and the Montage workflow, all based on real-world scientific computing practices, with their structures illustrated in Figure 3.

- *LIGO Workflow*: Originating from gravitational physics, this workflow aims to detect gravitational wave signals. After adjustments, the current version contains 40 task nodes, with added start and end task nodes (see Figure 3a).
- *Epigenomics Workflow*: Taking advantage of bioinformatics, it automates complex genomic sequence analysis and uses the MapReduce model concept, consisting of 10 task nodes (see Figure 3b).
- *CyberShake Workflow*: A project from the Southern California Seismic Network aimed at quantifying seismic impacts. Logically, it segments monitoring data to generate seismic maps. After adaptive improvements, it includes 17 task nodes (see Figure 3c).
- *Montage Workflow*: A large-scale astronomical image processing framework developed by NASA, which processes extensive image datasets through 14 task nodes for advanced image handling (see Figure 3d).

3.2. Basic DAG Model under Scheduling

The simple D model serves as a fundamental graph structure for task scheduling. In this model, each node represents a subtask, while each directed edge indicates a dependency from an upper node to a lower node. As the

name suggests, a DAG is a directed graph that contains no cycles, meaning each edge points from upper to lower nodes. If there is a directed edge between two nodes, it signifies a data transfer, with the edge's weight representing the size of the data. Consequently, a lower node must wait for all its preceding upper nodes to finish before it can execute. Each DAG includes a source node (with no incoming edges) and a termination node (with no outgoing edges). The depth level of each node is determined by its distance from the source node, counted by the number of edges in the shortest path.

As illustrated in Figure 4, node 0 is the source at level 0, nodes 1 to 4 are at level 1, nodes 5 to 8 are at level 2, and node 9 is the termination node at level 3. Except for the source node A and the termination node F , all other task nodes have parent and child nodes. The dependencies impose strict execution order constraints, meaning a task can only start after its preceding tasks are completed and after receiving necessary data from parent nodes.

Moreover, these dependencies require data transfer, which introduces overhead related to data size and the distribution of task nodes across servers, significantly impacting overall time consumption during data transfer. Previous research has not adequately considered these factors. Therefore, this paper addresses data transmission time as a key factor affecting platform duration, with specific quantification methods provided in Section 3.3.

Formally, a DAG workflow is represented by the triplet $G = (T, E, D)$, where $T = \{t_1, t_2, \dots, t_m\}$ is the set of subtasks (m being the number of subtasks). The dependencies between subtasks are denoted by $E = \{e_{ns} | t_n, t_s \in T\}$, where e_{ns} can be 0 or 1. If $e_{ns} = 1$, t_n is the parent node of t_s ; if $e_{ns} = 0$, there is no dependency. $D = \{d_{ns}\}$ represents the size of the resource data that task t_n depends on from task t_s , quantifying the data amount by assigning weights to the directed edges and the subtasks.

3.3. Workflow Model with Constraints

The proposed scheduling strategy for DAG workflow has a core optimization goal of minimizing the workflow scheduling time. Therefore, this section focuses on the internal connection that how the workflow scheduling time relates with the resource use efficiency, the data transmission status among subtasks, and the workflow execution makespan. And in terms of those factors, we designs a method designed to shorten the workflow scheduling time and execution makespan. At the same time, this strategy also constructs a complete set of task scheduling model. The establishment of this task scheduling model provides a solid foundation for the subsequent optimization of temporal scheduling in the context of multiple DAG workflows.

Different from definition above, let $G = (\mathcal{N}, \mathcal{E})$ represent a DAG consisting of a set of nodes denoted by $\mathcal{N}$ and a set of directed edges denoted by $\mathcal{E}$. Each node $i \in \mathcal{N}$ represents a task ("node" and "task" are used interchangeably in this paper), and each edge $i \rightarrow j$ means that task j can only begin after task i is completed and the required data transfer from i to j is completed.

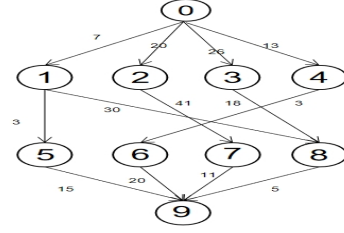


Figure 4: An example of DAG model where the numbered nodes have weights (not shown) to represent computational loads and the edges have weights to represent the data being transferred from source node to target node.

Given an edge from i to j , j is called the parent of i and i is called the child of j . A node without a parent node is called a source node; a node without a child node is called a sink node. It is easy to see that a DAG with multiple entry or exit nodes can be effectively converted to this standardized form.

Suppose there is a set of fully connected sets of heterogeneous resources denoted by R , and the time required for each unit of data to transfer from one resource to another, called transmission delay, is constant and is known in advance:

$$tl_{(i \rightarrow j), (p \rightarrow q)} = d_{i \rightarrow j} \times \eta_{p,q} \quad (1)$$

where, $d_{i \rightarrow j}$ represents the size of the transferred data from task i to task j , $\eta_{p,q}$ denotes the transmission rate between the resource p and q . Also assume that a resource can only run one task at a time, without considering preemption.

The expected Execution Time of task i with the resource $p \in R$ is modeled as a random variable $ET_{i,p}$. In the schedule of the DAG, each task i has a Start Time $ST_{i,p}$ and an Finish Time $FT_{i,p}$, p for the allocation of the resource p . Based on the constraints on the execution of a task of the DAG on a resource, it is clear that the following equations are satisfied:

$$ST_{j,q} = \max \left\{ FT_{l^*,q}, \max_{k \in \text{Par}(j)} \{ FT_{k,r(k)} + tl_{(k,r(k)) \rightarrow (j,q)} \} \right\} \quad (2)$$

where $FT_{l^*,q}$ represents the finish time of task l^* . The task is the last term on resource q Service, $\text{Par}(j)$ represents the set of all parent tasks of j , and $r(k)$ represents the task assigned, to which each task is assigned to a resource with the earliest completion time. So we have:

$$FT_{j,q} = ST_{j,q} + ET_{j,q} \quad (3)$$

And for the source node:

$$FT_{\text{entry_node}, r(\text{entry_node})} = ET_{\text{entry_node}, r(\text{entry_node})} \quad (4)$$

Furthermore, if no task is assigned to the resource q , then $FT_{l^*,q} = 0$. Due to the stochastic nature of the task execution time, the makespan is expressed as:

$$M = FT_{\text{exit_node}, r(\text{exit_node})} \quad (5)$$

Clearly, different schedules may lead to different M values. This paper notes that to better evaluate the approach, this paper assumes that data communication is predictable and only task execution time is modeled with random variables. This work can be easily extended to include data communication models based on random variables.

4. MCTS-based Scheduling Algorithm

The workflow scheduling problem is a well-known NP-hard problem, which means that under the current theoretical framework it is infeasible to expect to find a global optimal solution in heterogeneous distributed systems in polynomial time. Therefore, this paper utilizes heuristic algorithms (i.e. MCTS), to leverage its characteristic of randomized search in the global space, with the core objective of minimizing workflow makespan. The DAG-structured workflow scheduling problem is thus redefined as a normal task scheduling problem.

4.1. Task Preprocessing

Many widely-used heuristic scheduling strategies treat task execution times as constant values, determining them at the start of the scheduling phase. Even when task execution times follow probability distributions, static methods often extract a single statistical parameter, which may not yield optimal results, especially in multi-workflow environments where resource competition is significant. To address this, this paper proposes a DAG preprocessing scheme based on MCS techniques. This scheme aims to leverage the probabilistic distribution of task execution times rather than relying on a single estimate. It seeks to overcome the limitations of traditional static scheduling approaches by using simulation and sampling analysis to better understand the stochastic characteristics of task execution times. The goal is to achieve improved scheduling strategies and overall performance optimization in the decision-making process.

We denoted a DAG as G , which is assigned to a specific set of resources R . The operation can be broken down into the following steps:

The input space for preprocessing I is constituted by the predicted random task execution times of a given application, denoted as:

$$I_G = \{T_{i,p} | i \in N, p \in R\} \quad (6)$$

where N represents the number of tasks, R symbolizes the number of available resources, and $T_{i,p}$ denotes task i expectations on resource p execution time.

Random sampling: in the MCTS, the sampling process through the function from the input space f_{smp} implementation, as shown below,

$$P_G = f_{smp}(I_G) = \{t_{i,p} | i \in N, p \in R\} \quad (7)$$

where, P_G is regarded as a set of static G task execution time prediction, $t_{i,p}$ is $T_{i,p}$ random sampling values. In addition,

Algorithm 1 Preprocessing Algorithm

Input: A DAG G with prediction information I_G ;

Output: A list L_Ω of Ω_G .

```

1: while Generation step of  $H$  is not finished do
2:    $P_G \leftarrow \{t_{i,p} | i \in N, p \in R\};$  ▷ Formula (7)
3:    $\Omega_G \leftarrow \text{StaticSched}_H(G, P_G);$  ▷ Formula (9)
4:   if  $\Omega_G$  is a new static schedule then
5:      $L_\Omega \leftarrow \Omega_G;$ 
6:   end if
7: end while
8: return  $L_\Omega$ .
```

we define an estimated set $\bar{P}_G$, which contains the expected value of the task execution time, expressed as follows.

$$\bar{P}_G = \{\mu_{i,p} | i \in N, p \in R\} \quad (8)$$

where $\mu_{i,p}$ represents the expected execution time of task i with resource p .

We defines two deterministic calculation methods. The first is to calculate static timetable. Given a DAG G and an associated set of static predictions P_G , any static scheduling heuristic H (HEFT in this paper) can be used to generate a static schedule Ω_G , denoted as follows,

$$\Omega_g = \text{StaticSched}_H(G, P_G) \quad (9)$$

More specifically, the average static schedule can be obtained in this paper by applying H to $\bar{P}_G$ as follows,

$$\bar{\Omega}_G = \text{StaticSched}_H(G, \bar{P}_G) \quad (10)$$

The second is calculated under the assumption that the exact execution time of each task on each resource P_G^* under the condition of known, calculate the static time span generated schedule. This calculation can be represented as:

$$m = \text{Makespan}(G, P_G^*, \Omega_G) \quad (11)$$

The core of the preprocessing phase is construction. During the construction stage, the system first extracts a certain number of samples from the input space and employs a specific static scheduling heuristic algorithm (such as CPOP or other similar strategies) to generate a series of diversified and static scheduling schemes based on these samples. The preprocessing algorithm is illustrated in Algorithm 1. It is important to emphasize that the time complexity of heuristic algorithms (like CPOP) is typically $\mathcal{O}(e \times p)$, where e represents the number of edges in the directed acyclic graph (DAG), and p denotes the number of available resources.

4.2. Algorithm Design

We pre-processed workflow scheduling on DAG in Section 4.1. And we obtained many cases of different critical path through CPOP algorithm. We will evaluate the critical path and choose the shortest critical path. In this paper, we consider to use MCTS method to work with next step.

For the search tree, this paper defines its root node as t_0 , with the action executed as a . The visit count is initialized

as $N(t_0, a) = 0$, and the reward value is initialized as $Q(t_0, a) = 0$. Next, a to-do list is established to store tasks that need to be scheduled. During the scheduling process, the tasks in this to-do list will continuously change. The algorithm proposed is illustrated in Algorithm 2.

The MCTS process is divided into four stages, which will be explained sequentially.

Selection: Starting from t_0 , recursively select the child node t with the highest UCB value until a terminal node is reached.

Extension: If the selected leaf node is not a terminal state, a new set of child nodes is generated based on the principle of $q \times T$, initializing their visit counts and reward values. The information of these newly generated child nodes is integrated into the search tree structure, where the branches of the tree carry information about tasks and processors, while the nodes themselves record their visit counts and corresponding reward values. Then, one node t_s is randomly selected from this set of child nodes for the next operation.

Simulation: The selected expansion node t_s serves as the starting point for a dedicated strategy mechanism for task selection and processor allocation. First, a random strategy is used to select tasks in the DAG that have not yet been scheduled. Then, for each selected task, its earliest start time (EST) and latest finish time (EFT) are calculated. Next, a processor is chosen, setting a threshold R^* and adopting a mixed strategy for decision-making: for each task, with a probability of R^* , a greedy strategy is used to select the optimal processor, while with a probability of $1 - R^*$, a random strategy is employed to select a processor. This process continues until all tasks are successfully allocated to appropriate processors. Ultimately, based on the scheduling scheme, the overall completion time—i.e., makespan—is obtained.

Backpropagation: Finally, based on the makespan value, the latest effective node information obtained from the simulation is fed back from the bottom up to the root node to update the search tree accordingly. Specifically, for the visited nodes and their corresponding actions, the update of visit counts follows an incremental rule, expressed as $N(t, a) = N(t, a) + 1$.

For the reward value update of the nodes, adjustments are made based on the makespan value obtained from this simulation using the following update formula: $Q(t, a) = Q(t, a) + \text{makespan}$. This means that after each simulation, the cumulative reward for the node will be incremented according to the makespan result from this scheduling.

This process iterates through the four key steps until the simulation reaches a preset maximum number of iterations or satisfies other stopping conditions. At the end of the simulation, the MCTS task scheduling algorithm records the minimum makespan value observed throughout all iterations, which will be regarded as the result of the optimal scheduling strategy.

The overall computational complexity of the proposed method is influenced by two factors: the complexity of the

Algorithm 2 MCTS Scheduling Algorithm (MSA)

Input: A DAG G with prediction information I_G .

Output: The best schedule $\Omega_{G_{best}}$ for G .

```

1:  $m_{best} \leftarrow +\infty$ ;
2:  $\Omega_{G_{best}} \leftarrow NULL$ 
3:  $L_\Omega \leftarrow \text{Preprocessing}(G, I_G)$  ▷ Algorithm 1
4: for Selection rules of  $H$  are satisfied do
5:    $\bar{P}_G \leftarrow \{\mu_{i,p} | i \in N, p \in R\}$ ; ▷ Formula (8)
6:   for  $\Omega_G$  in  $L_\Omega$  do
7:      $P_G^* \leftarrow$  accurate execution time;
8:     Assume  $\bar{P}_G = P_G^*$ ;
9:      $m \leftarrow \text{Makespan}(G, P_G^*, \Omega_G)$ ; ▷ Formula (11)
10:    if  $m < m_{best}$  then
11:       $m_{best} \leftarrow m$ ;
12:       $\Omega_{G_{best}} \leftarrow \Omega_G$ 
13:    end if
14:  end for
15: end for
16: return  $\Omega_{G_{best}}$ .
```

scheduling heuristic used in the production phase and the complexity of functions in the selection phase. For a DAG with v vertices and e edges scheduled on p resources, the production phase uses HEFT, which has a time complexity of $O(e \times p)$. The selection phase has a complexity of $O(e)$. Thus, the overall complexity of the MCS method is expressed as $O(e \times p \times a + e \times b)$. In practical scenarios, with n fixed, this simplifies to $O(e \times p \times a)$. If the DAG is dense, where the number of edges is proportional to the square of the number of vertices ($O(v^2)$), the complexity increases to $O(v^2 \times p \times a)$.

5. Experimental Evaluations

We implemented scDAG based on for the sake of experimental evaluation. We will introduce parameters of our experimental environment and analytical performance of scDAG based on multi-science. Agency using experimental environment parameters, then to the whole scDAG system performance analysis, test it in a variety of scenarios of performance.

5.1. Experimental Setup

CloudSim [27] is a widely accepted cloud computing simulation tool in both academia and industry, utilized by researchers to validate and test various task scheduling algorithms and virtual machine deployment strategies. It collects performance metrics like runtime and energy consumption for analysis.

We developed the scDAG prototype system based on CloudSim, enhancing it with additional functionalities. We built task modules to include resource requirements and implemented virtualization resource management for effective resource allocation between hosts and virtual machines. Interfaces for resource allocation strategies were also created.

TABLE 1: Configurations of our experimental setup

Setup	Configurations
CPU core	Intel(R) Xeon(R) CPU E5-2630 v4 @ 2.20GHz (x40)
Memory	DDR4 ECC 64GB
Ethernet card	Intel Corporation 82599ES 10-Gigabit
OS	Ubuntu 14.04.6 LTS
Linux version	4.4.0-31-generic

Overall, scDAG improves the reliability and practicality of simulation experiments and provides a dynamic interactive mode for testing scheduling strategies. The specific experimental environment is detailed in Table 1.

5.2. Workflows in Practice

The experimental design employs three types of DAGs, abstracted from real-world scientific computing scenarios, to represent scheduled workflows: the LIGO scientific workflow [28] from gravitational physics, comprising up to 40 scientific task nodes; the CyberShake workflow [29], widely used in seismic research, made up of 17 interconnected task nodes; and the Montage scientific workflow [30] in astronomy, organized with 14 task nodes.

To test the proposed energy-saving scheduling strategy across diverse scenarios, we mixed these workflows in a 1:1:1 ratio. For a total of 600 workflow instances, each type contributed 200 instances. This mixed synchronous scheduling approach effectively demonstrates the strategy's compatibility and versatility across different scales and fields.

5.3. Workflow Parameter Generation

Scientific workflows in practice provide a robust DAG structure, making it reasonable and credible to conduct experiments based on these real workflow structures. To assess the relative performance of heuristic algorithms, this paper defines the shape and structure of the DAG through six parameters and generates random DAGs. These parameters are as follows:

Number of nodes (NN): DAG counts the number of compute nodes (that is, the application of task number);

Communication ratio (RR): This is the ratio of the average communication between nodes in the DAG to the average computation by each node. A low computing-to-communication ratio indicates a computationally intensive DAG, while a high ratio suggests it is communication intensive.

Height and width (HW): These parameters influence the DAG's height and width, affecting its degree of parallelism. A low height-to-width ratio results in a wider DAG, while a high ratio makes the DAG appear longer.

Density: This measures the number of edges between two levels in the DAG. A low density results in fewer edges, while a high density increases the number of edges.

Regularity: Regularity indicates the consistency of task numbers across levels. Low values signify varying task

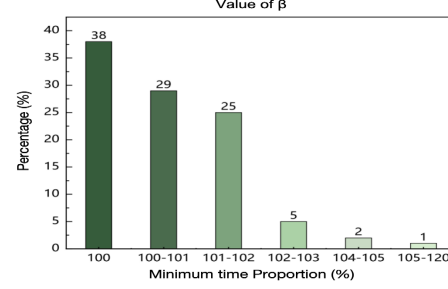


Figure 5: Screening condition experiments

counts at different levels, while high values indicate a similar number of tasks at each level.

Jump: This refers to edges that can connect from level l to level $l + jump$. $jump = 1$ means a connection between two consecutive levels.

The parameters are set as follows:

$$SET_{NN} = \{10, 20, 30, 40, 50, 60, 70, 80, 90, 100\},$$

$$SET_{RR} = \{1.0, 5.0, 10.0\},$$

$$SET_{HW} = \{0.4, 0.6\},$$

$$SET_{Jump} = \{1, 2, 3\},$$

$$SET_{Density} = \{0.2, 0.4\},$$

$$SET_{Regularity} = \{0.4, 0.8\},$$

$$SET_{Processors} = \{3, 4, 5\}.$$

Different combinations of the above parameters provide 2160 different types of DAG. With 10 DAGs randomly generated by each type of DAG, the sample space of DAG is about 21K. For the sake of fairness, random search algorithm and WSA have the same running time in our experiments.

5.4. Experimental Analysis

In this subsection, we specifically illustrate the threshold method mentioned in Section 4.2. Then we introduce QoE to describe degree of deviation between average execution makespan and actual execution makespan, and then research performance of the MCTS method on different values of QoE. Finally, we evaluate the performance of our algorithm with random algorithm, HEFT and CPOP.

Threshold Method for MSA. In Simulation Step described in Section 4.2, this paper proposes a threshold method to exclude certain scheduling schemes generated during the simulation stage, reducing evaluation costs in the filtering phase. The core challenge is determining the appropriate threshold level. The paper follows these steps:

- 1) Select a representative DAG instance from various types and resource configurations.
- 2) Implement MCS on the selected DAG without threshold filtering.

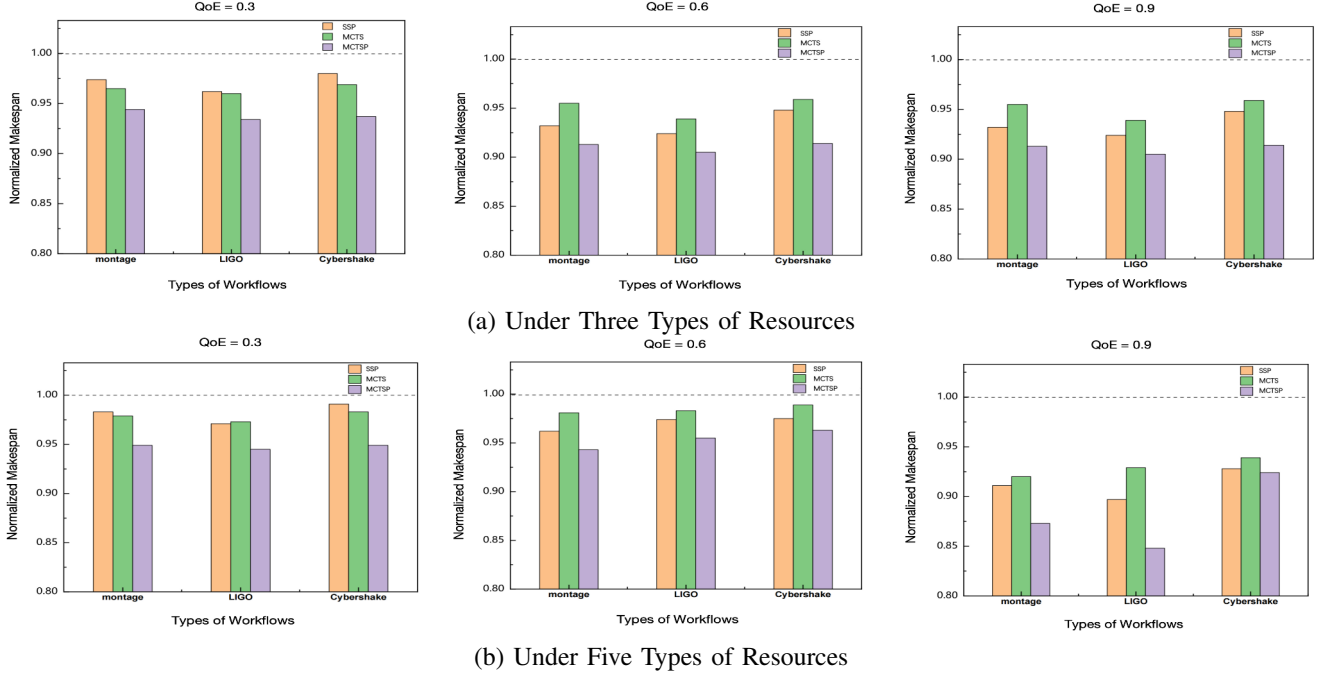


Figure 6: The execution makespans under different QoE for different amount of types of resources where SSP means static scheduling with preprocessing, MCTSP represents MCTS with preprocessing(i.e., MSA).

- 3) Record the maximum execution makespan of each independent schedule plan based on actual task durations.
- 4) Identify the minimum normalized makespan among all generated plans.
- 5) In the filtering phase, select an optimal scheduling scheme.
- 6) Calculate the normalized makespan of the selected scheme and denote the difference from the minimum as variable β .
- 7) Repeat steps 1 to 6 a total of 1000 times to analyze the distribution characteristics of β .

Figure 5 shows the distribution of β values after 1000 iterations, categorized into eight intervals: (i) 0, (ii) greater than 0 and up to 1, (iii) 1% to 2%, (iv) 2% to 3%, (v) 3% to 4%, (vi) 4% to 5%, and (vii) 5% to 20%. The data indicates that over 91% of the 1000 trials had a normalized makespan within 1.02 times the minimum. Consequently, the paper sets a threshold of 0.02, excluding any normalized makespan exceeding this value during production. Although the exact minimum makespan is unknown beforehand, a dynamic approach is used to continuously update this threshold based on the shortest estimated makespan as production progresses.

Quality of Makespan (QoE) of MSA. Assuming that the actual task execution time follows a probability distribution around a given average execution time (denoted as μ) and varies within a certain range, the paper establishes a boundary for the actual execution time after determining

the average execution time.

Similar to the concept of QoI proposed in Casanova's work, this paper introduces QoE, denoted by the symbol $0 < \delta < 1$, to describe the maximum percentage deviation limit of the actual task execution time relative to its mean (where $\delta = 0$ indicates no deviation). For task execution times following a normal distribution, the upper and lower bounds are set at $\pm 3\sigma$ around the mean μ , where a single standard deviation is equal to $\frac{\mu \cdot \delta}{3}$. For uniform distribution, the boundaries are set between $\mu \cdot (1 - \delta)$ and $\mu \cdot (1 + \delta)$. This process is repeated for each task and its corresponding resources.

In the experimental phase, this paper considers three different QoE values: 0.3, 0.6, and 0.9 under different amounts of types of resources. We calculate the average maximum makespan of the DAG for each method. This is achieved by randomly predicting task execution times 100 times and repeating the generation process 50 times (thus, the average Makespan is based on over 5000 independent values).

As shown in Figure 6, MCTS performs moderately across different QoE levels. However, when combined with a preprocessing algorithm (WSSA), it significantly reduces makespan compared to static algorithms, particularly when QoE is below 0.6, indicating the reliability of our preprocessing algorithm in enhancing MCTS's search for better scheduling solutions. Additionally, the algorithm performs better in resource-constrained situations (with three resource types). This is reasonable, as abundant resources increase the search space, making it harder for MCTS to find good

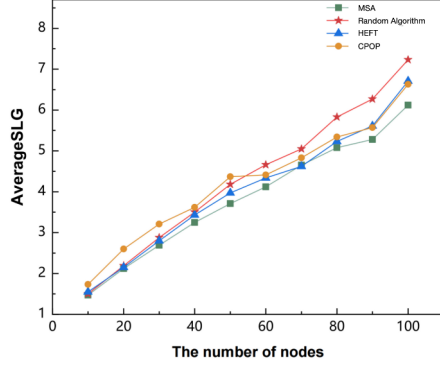


Figure 7: Average SLR of different methods

solutions quickly.

Performance of MSA Compared with Other Algorithms. In order to explore the effectiveness of the proposed method, we compared our algorithm with HEFT and CPOP to explore the relationship between performance of algorithms and characteristic of DAGs. We apply schedule length ratio (SLR) and ratio of scheduling time with all subtasks serialized and scheduling time with all subtasks parallelized (Speedup). Here we have several definitions:

- *SLR*: With different sizes of DAGs is, it is necessary to use the ratio of scheduling length and theoretical scheduling time for the sake of normalized scheduling length. SLR is defined as:

$$SLR = \frac{\sum_{i \in CPMIN} m_i}{\sum_{j \in P} \min_{i \in CP} w(i, j)},$$

where $\sum_{j \in P} \min_{i \in CP} w(i, j)$ is the sum of execution time of all subtasks on critical path by the fastest processor, which is a lower bound of scheduling time. SLR is not less than 1 according to its definition and a smaller SLR means a shorter scheduling time and better performance.

- *Speedup*: Speedup represents the ratio of scheduling time with all subtasks serialized and scheduling time with all subtasks parallelized which is defined as:

$$Speedup = \frac{\min_{p_j \in P} \left[\sum_{i \in V} w(i, j) \right]}{makespan(solution)},$$

- *Running time*: Running time of algorithm is the time cost by generating a schedule (i.e., a sequence of subtasks) given a specific DAG.

Figure 7 illustrates the relationship between SLR and DAG size for different algorithms. A lower SLR indicates better scheduling performance. Our algorithm performs similarly, or even better, than others; for instance, with a DAG of 10 tasks, HEFT and CPOP yield average SLRs of 1.56 and 1.39, respectively, while our algorithm reduces the average SLR by about 8 percentage points. Figure 8 shows the average running time of different algorithms. Our algorithm

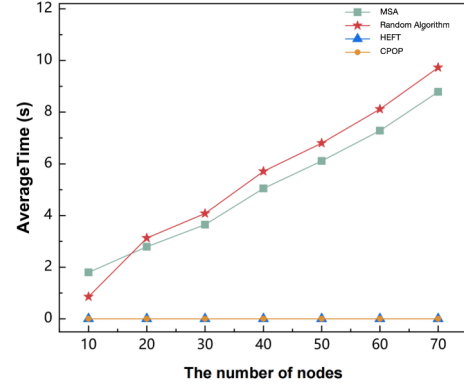


Figure 8: Average running time of different methods

demonstrates lower average running time, indicating better efficiency. When the number of subtasks is less than 20, the smaller search space allows for quicker scheduling. Overall, our algorithm consistently outperforms others in both SLR and speedup across various DAG sizes.

6. Conclusion

With the continuous evolution of internet technology, the scale of data computation is growing rapidly, and the emergence of heterogeneous distributed systems alleviates the bottleneck of computing power. However, efficient task allocation and scheduling in this new environment have become central research challenges. This paper addresses these challenges by aiming to allocate suitable resources to different processors, thereby enhancing parallelization and resource utilization. We model workflows in heterogeneous distributed environments as DAGs and propose a novel two-phase scheduling algorithm that optimizes task execution order using MCTS. Additionally, we developed a prototype system named scDAG based on the CloudSim framework, which implements functional enhancements for task modeling, resource management, and scheduling strategies. Experimental evaluations on the scDAG platform demonstrate that our assumption of leveraging average execution makespan is both feasible and effective. Due to the representativeness and reliability of the HEFT and CPOP algorithms, our method exhibits superior performance compared to these classic approaches. The results confirm that our model and algorithm are robust and effective. This work validates the applicability of the research findings in efficiently scheduling DAG tasks, providing an important theoretical foundation and practical platform for studying task scheduling strategies in heterogeneous distributed environments. MCTS shows great potential in scheduling, and future work will further explore parallel computing directions and attempt to apply MCTS to address challenges such as lock-free and wait-free scenarios under resource constraints.

Acknowledgments

This work is supported in part by the 3rd Xinjiang Scientific Expedition Program(2021xjkk1300), the SIAT-SunTang Big Data&AI Joint Innovation Laboratory (E3Z092), Shenzhen Science and Technology Plan Project (Shenzhen-Hong Kong-Macau Category C, No. SGDX20220530111001003), and Shenzhen Science and Technology Program (CJGJZD20230724093659004). Some grammatical errors are corrected using ChatGPT.

References

- [1] X. Gao, R. Liu, and A. Kaushik, "Hierarchical multi-agent optimization for resource allocation in cloud computing," *IEEE Transactions on Parallel and Distributed Systems*, vol. 32, no. 3, pp. 692–707, 2021.
- [2] P. Berenbrink, C. Cooper, T. Friedetzky, T. Friedrich, and T. Sauerwald, "Randomized diffusion for indivisible loads," *Journal of Computer and System Sciences*, vol. 81, no. 1, pp. 159–185, 2015. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S0022000014000713>
- [3] S. Wang, X. Li, Q. Z. Sheng, R. Ruiz, J. Zhang, and A. Beheshti, "Multi-queue request scheduling for profit maximization in iaaS clouds," *IEEE Transactions on Parallel and Distributed Systems*, vol. 32, no. 11, pp. 2838–2851, 2021.
- [4] M. A. Rodriguez and R. Buyya, "Deadline based resource provisioning and scheduling algorithm for scientific workflows on clouds," *IEEE Transactions on Cloud Computing*, vol. 2, no. 2, pp. 222–235, 2014.
- [5] C. E. Clark, "Monte carlo," *Commun. ACM*, vol. 4, no. 3, p. 141, mar 1961. [Online]. Available: <https://doi.org/10.1145/366199.366247>
- [6] G. He, Z. Singh, and E. Yoneki, "Mcts-geb: Monte carlo tree search is a good e-graph builder," p. 26–33, 2023. [Online]. Available: <https://doi.org/10.1145/3578356.3592577>
- [7] H. Topcuoglu, S. Hariri, and M.-Y. Wu, "Performance-effective and low-complexity task scheduling for heterogeneous computing," *IEEE Transactions on Parallel and Distributed Systems*, vol. 13, no. 3, pp. 260–274, 2002.
- [8] F.-H. Liu, H.-H. Liu, and P. W. H. Wong, "Greedy is optimal for online restricted assignment and smart grid scheduling for unit size jobs," *Theor. Comp. Sys.*, vol. 65, no. 6, p. 1009–1032, aug 2021. [Online]. Available: <https://doi.org/10.1007/s00224-021-10037-w>
- [9] J. Li, H. Xu, Y. Zhu, Z. Liu, C. Guo, and C. Wang, "Aryl: An elastic cluster scheduler for deep learning," *CoRR*, vol. abs/2202.07896, 2022. [Online]. Available: <https://arxiv.org/abs/2202.07896>
- [10] Y. Yang, K. Wang, G. Zhang, X. Chen, X. Luo, and M.-T. Zhou, "Meets: Maximal energy efficient task scheduling in homogeneous fog networks," *IEEE Internet of Things Journal*, vol. 5, no. 5, pp. 4076–4087, 2018.
- [11] S. Nabi, M. Ibrahim, and J. M. Jimenez, "Dralba: Dynamic and resource aware load balanced scheduling approach for cloud computing," *IEEE Access*, vol. 9, pp. 61 283–61 297, 2021.
- [12] M. C. Calzarossa, M. L. D. Vedova, L. Massari, G. Nebbione, and D. Tessera, "Multi-objective optimization of deadline and budget-aware workflow scheduling in uncertain clouds," *IEEE Access*, vol. 9, pp. 89 891–89 905, 2021.
- [13] D. Huff, S. Dai, and P. Hanrahan, "Clockwork: Resource-efficient static scheduling for multi-rate image processing applications on fpgas," in *2021 IEEE 29th Annual International Symposium on Field-Programmable Custom Computing Machines (FCCM)*, 2021, pp. 186–194.
- [14] G. Zhou, Z. Chen, C. Zhang, and F. Chang, "An adaptive ensemble deep forest based dynamic scheduling strategy for low carbon flexible job shop under recessive disturbance," *Journal of Cleaner Production*, vol. 337, p. 130541, 2022. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S0959652622001834>
- [15] F. Zhang, Y. Mei, S. Nguyen, K. C. Tan, and M. Zhang, "Multitask genetic programming-based generative hyperheuristics: A case study in dynamic scheduling," *IEEE Transactions on Cybernetics*, vol. 52, no. 10, pp. 10 515–10 528, 2022.
- [16] H. Arabnejad and J. G. Barbosa, "List scheduling algorithm for heterogeneous systems by an optimistic cost table," *IEEE Transactions on Parallel & Distributed Systems*, vol. 25, no. 3, pp. 682–694, 2014.
- [17] D. Eddy and M. Kochenderfer, "Markov decision processes for multi-objective satellite task planning," in *2020 IEEE Aerospace Conference*, 2020, pp. 1–12.
- [18] C. Yanqiu, H. Xianbiao, T. Qing, Q. Hongsheng, and Y. Hong, "Monte carlo tree search-based mixed traffic flow control algorithm for arterial intersections," 2020.
- [19] C. Yoo, S. E. Lensgraf, R. C. Fitch, L. M. Clemon, and R. R. Mettu, "Toward optimal fdm toolpath planning with monte carlo tree search," *2020 IEEE International Conference on Robotics and Automation (ICRA)*, pp. 4037–4043, 2020. [Online]. Available: <https://api.semanticscholar.org/CorpusID:211032148>
- [20] A. Haj-Ali, H. Genç, Q. Huang, W. S. Moses, J. Wawrzyniek, K. Asanović, and I. Stoica, "Protuner: Tuning programs with monte carlo tree search," *ArXiv*, vol. abs/2005.13685, 2020. [Online]. Available: <https://api.semanticscholar.org/CorpusID:218972073>
- [21] K. Li, Q. Deng, L. Zhang, Q. Fan, G. Gong, and S. Ding, "An effective mcts-based algorithm for minimizing makespan in dynamic flexible job shop scheduling problem," *Computers Industrial Engineering*, vol. 155, p. 107211, 2021. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S0360835221001157>
- [22] H. Shuai and H. He, "Online scheduling of a residential microgrid via monte-carlo tree search and a learned model," *IEEE Transactions on Smart Grid*, vol. 12, no. 2, pp. 1073–1087, 2021.
- [23] M. Hoffman, E. Song, M. P. Brundage, and S. Kumara, "Online improvement of condition-based maintenance policy via monte carlo tree search," *IEEE Transactions on Automation Science and Engineering*, vol. 19, no. 3, pp. 2540–2551, 2022.
- [24] J. Oren, C. Ross, M. Lefarov, F. Richter, A. Taitler, Z. Feldman, C. Daniel, and D. D. Castro, "Solo: Search online, learn offline for combinatorial optimization problems," in *Symposium on Combinatorial Search*, 2021. [Online]. Available: <https://api.semanticscholar.org/CorpusID:233024705>
- [25] S. Tuli, G. Casale, and N. R. Jennings, "Mcds: Ai augmented workflow scheduling in mobile edge cloud computing systems," *IEEE Transactions on Parallel and Distributed Systems*, vol. 33, no. 11, pp. 2794–2807, 2022.
- [26] S.-J. Y. Hok-Leung Kung and K.-C. Huang, "An improved monte carlo tree search approach to workflow scheduling," *Connection Science*, vol. 34, no. 1, pp. 1221–1251, 2022.
- [27] R. N. Calheiros, R. Ranjan, A. Beloglazov, C. A. F. D. Rose, and R. Buyya, "Cloudsim: a toolkit for modeling and simulation of cloud computing environments and evaluation of resource provisioning algorithms," *Software: Practice and Experience*, no. 1, 2011.
- [28] A. Abramovici, W. E. Althouse, R. W. P. Drever, Y. Gürsel, and M. E. Zucker, "Ligo: The laser interferometer gravitational-wave observatory," *Science*, vol. 256, no. 5055, pp. 325–333, 1992.
- [29] R. Graves, T. H. Jordan, S. Callaghan, E. Deelman, E. Field, G. Juve, C. Kesselman, P. Maechling, G. Mehta, and K. a. Milner, "Cyber-shake: A physics-based seismic hazard model for southern california," *Pure and Applied Geophysics*, vol. 168, no. 3–4, pp. 367–381, 2011.
- [30] A. Laitya, T. A. Princed, G. Singhb, and M. H. Sub, "Montage: A grid enabled engine for delivering custom science-grade mosaics on demand g. bruce berriman†a, ewa deelmanb , john gooda, joseph jacobc, daniel s. katzc, carl kesselmanc.,"